



Greedy Problem Solving 2

- Gaurish Baliga



Problem 1: Duff and Weight Lifting

Problem Link: <https://codeforces.com/problemset/problem/587/A>

Recently, Duff has been practicing weight lifting. As a hard practice, Malek gave her a task. He gave her a sequence of weights. Weight of i -th of them is 2^{w_i} pounds. In each step, Duff can lift some of the remaining weights and throw them away. She does this until there's no more weight left. Malek asked her to minimize the number of steps.



Duff is a competitive programming fan. That's why in each step, she can only lift and throw away a sequence of weights $2^{a_1}, \dots, 2^{a_k}$ if and only if there exists a non-negative integer x such that $2^{a_1} + 2^{a_2} + \dots + 2^{a_k} = 2^x$, i. e. the sum of those numbers is a power of two.

Duff is a competitive programming fan, but not a programmer. That's why she asked for your help. Help her minimize the number of steps.



Problem 2: Duff and Weight Lifting

Examples

input	Copy
5 1 1 2 3 3	
output	Copy
2	

input	Copy
4 0 1 2 3	
output	Copy
4	

Note

In the first sample case: One optimal way would be to throw away the first three in the first step and the rest in the second step. Also, it's not possible to do it in one step because their sum is not a power of two.

In the second sample case: The only optimal way is to throw away one weight in each step. It's not possible to do it in less than 4 steps because there's no subset of weights with more than one weight and sum equal to a power of two.

Obs 1: Weights are powers of two

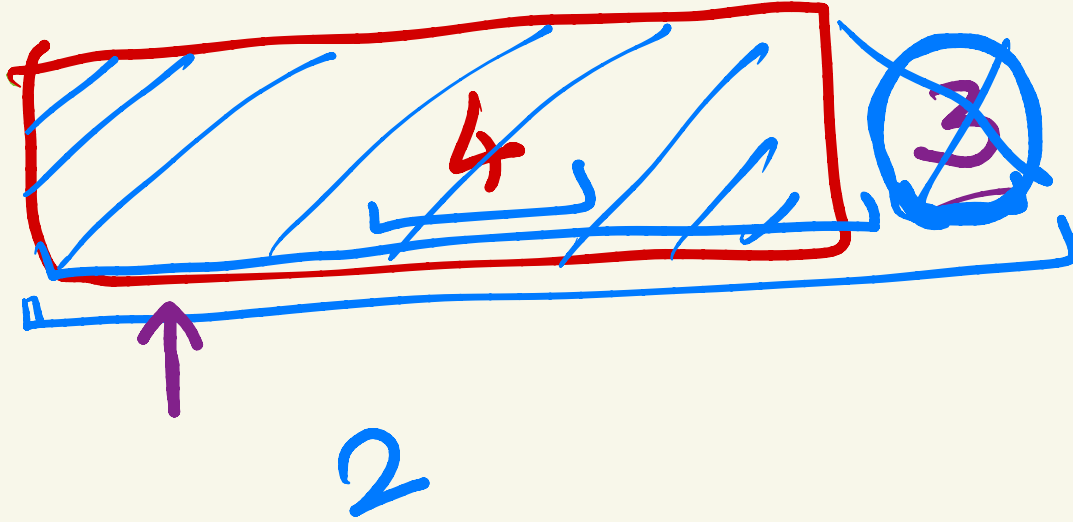
$\sum_{i \in S} 2^i$ should be a power of two



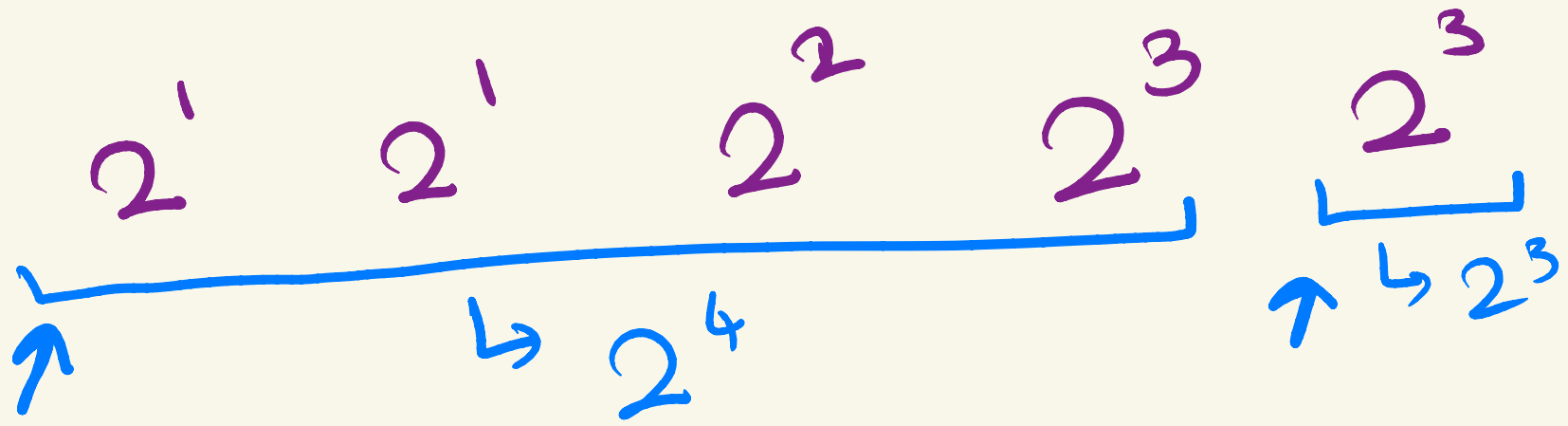
$\{2^x, 2^y, 2^z\} \rightarrow \Sigma$ is a
power of
two

smallest power
occurs at least twice

→ Min possible moves



$O(n \log n)$





Some observations:

1. We want to club up as many numbers together as possible such that their sum is a power of two.
2. Consider the smallest number (2^a) in a sequence $2^a, 2^b, 2^c$.
3. We can claim that 2^a should occur atleast one more time if it is part of a larger sequence.
4. Why? Because 2^a can be only clubbed with 2^a to make a power of 2.



Solution

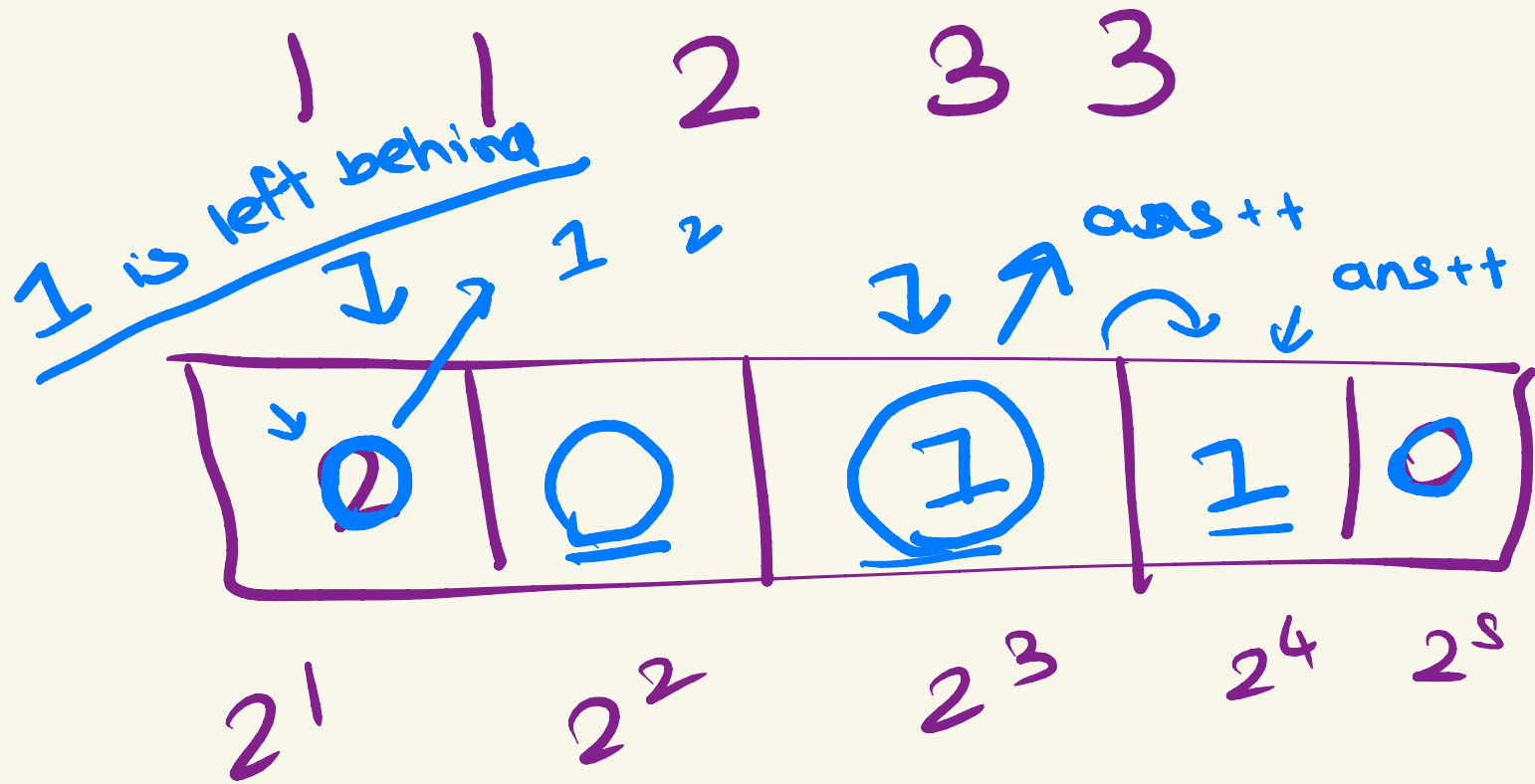
1. Note down the frequency of each power of two.
2. Start from the smallest power of 2 and iterate towards the largest.
3. If you have x elements = 2^i , then club most of them to form $\text{floor}(x / 2)$ elements = $2^{(i + 1)}$ and you might and might not have a remainder left.
4. If remainder is left ($x \% 2 == 1$) that means that this element needs to be handled separately.

4 1^s $\longrightarrow$ 2 2^2

5 1^s $\longrightarrow$ 2 $2s$

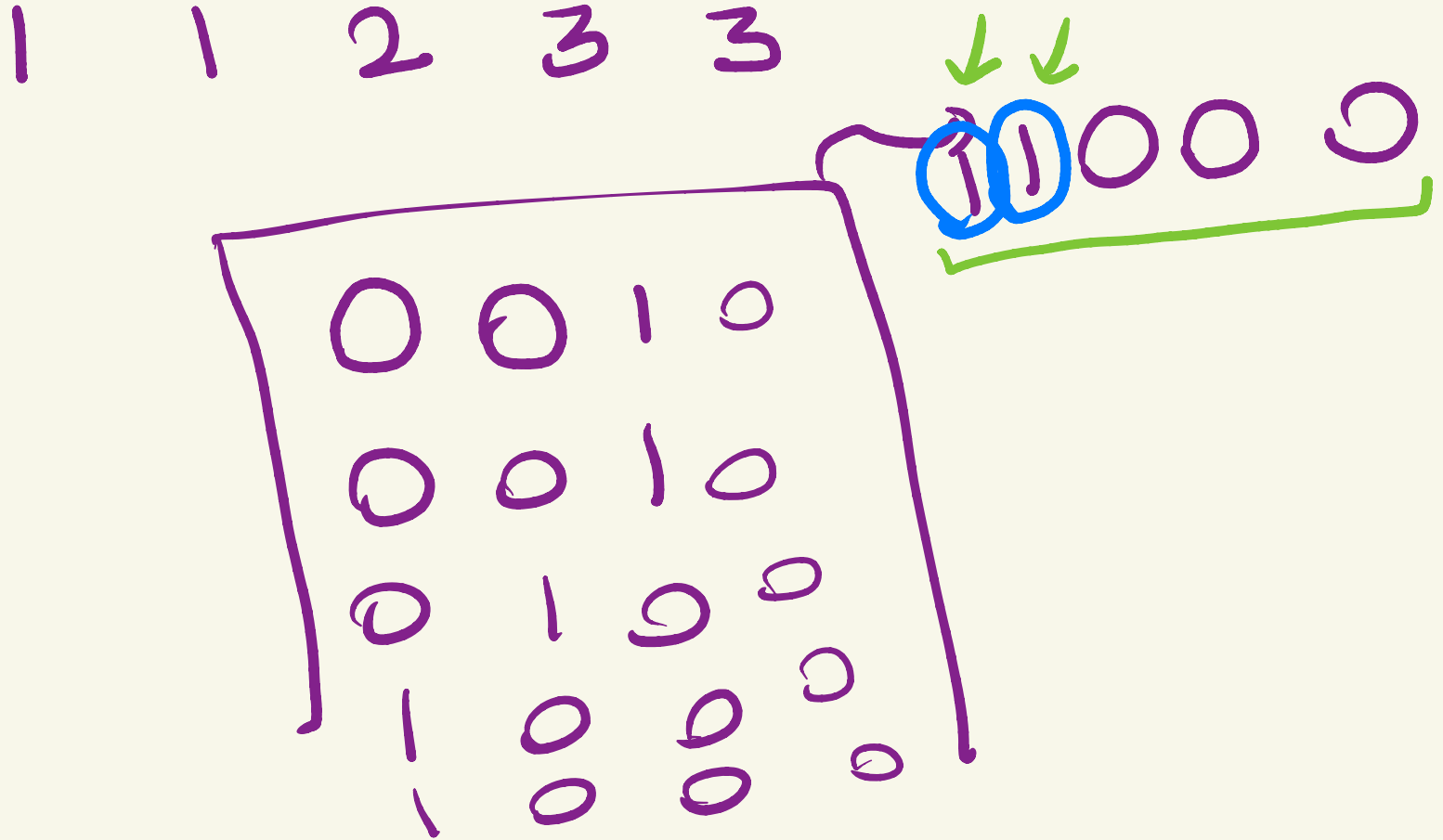
1 1

P9



answer ++;

2





Implementation

```
int n, answer = 0; cin >> n;

for(int i = 0; i < n; i++) {
    int x; cin >> x;
    f[x]++;
}

for(int i = 0; i + 1 < N; i++) {
    f[i + 1] += f[i] / 2; ← carry
    f[i] &= 1;
    answer += f[i];
}

cout << answer << "\n";
```

Problem 2: Polycarp at the Radio



Problem Link: <https://codeforces.com/problemset/problem/723/C>

Polycarp is a music editor at the radio station. He received a playlist for tomorrow, that can be represented as a sequence $a_1, a_2, \dots, a_n$, where a_i is a band, which performs the i -th song. Polycarp likes bands with the numbers from 1 to m , but he doesn't really like others.

We define as b_j the number of songs the group j is going to perform tomorrow. Polycarp wants to change the playlist in such a way that the minimum among the numbers $b_1, b_2, \dots, b_m$ will be as large as possible.

Find this maximum possible value of the minimum among the b_j ($1 \leq j \leq m$), and the minimum number of changes in the playlist Polycarp needs to make to achieve it. One change in the playlist is a replacement of the performer of the i -th song with any other group.

Input

The first line of the input contains two integers n and m ($1 \leq m \leq n \leq 2000$).

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$), where a_i is the performer of the i -th song.



Problem 2: Polycarp at the Radio

input	<u>3</u>	Copy
6 1		
1 2 3 2 2 2		
output		Copy
2 1		
1 2 1 2		

input		Copy
7 3		
1 3 2 2 2 2 1		
output		Copy
2 1		
1 3 3 2 2 2 1		

Note

In the first sample, after Polycarp's changes the first band performs two songs ($b_1 = 2$), and the second band also performs two songs ($b_2 = 2$). Thus, the minimum of these values equals to 2. It is impossible to achieve a higher minimum value by any changes in the playlist.

In the second sample, after Polycarp's changes the first band performs two songs ($b_1 = 2$), the second band performs three songs ($b_2 = 3$), and the third band also performs two songs ($b_3 = 2$). Thus, the best minimum value is 2.

1. Using a pq of frequencies

↳ impl- complicated

max min freq.

$x-1$

x

$x+1$

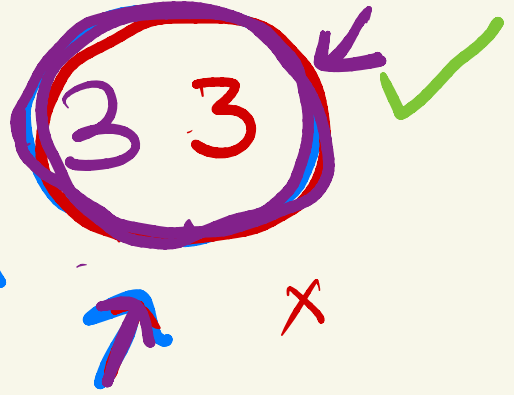
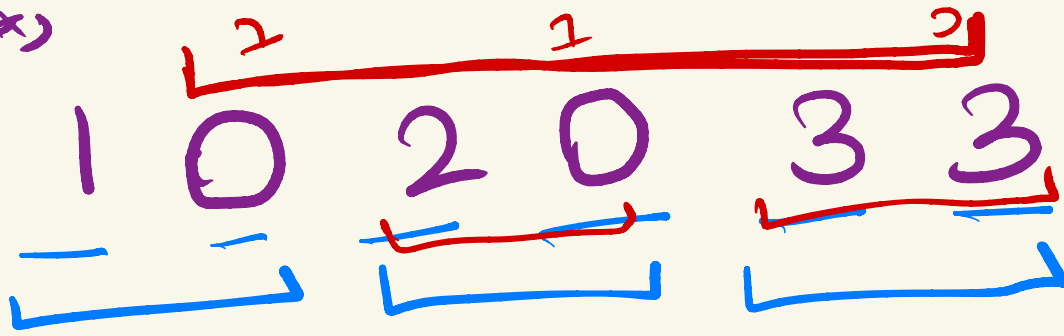
X

x

false

Binary Search

2 elements



$$x = 2$$

$$m = 3$$

$$2 \quad 3s$$

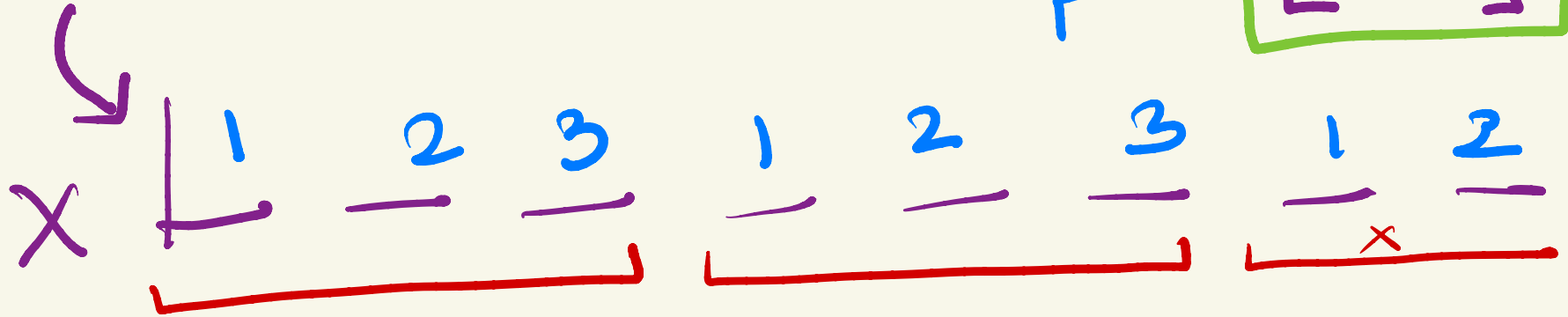
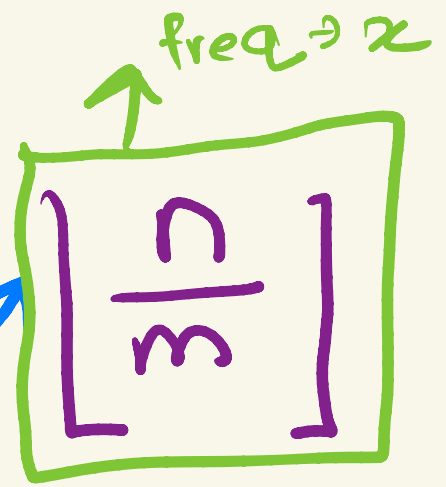
$> m$ taken

Elegant Obs.

$$1 - m$$

$$m = 3$$

$$n = 8$$





Some observations:

1. **Trivial Observation:** Binary Search on Answer is possible
2. **Non-trivial observation:** Each element from 1 to m can occur atleast $\text{floor}(n / m)$ times.



Solution

1. The min max frequency is just n / m .
2. Now the main task is to implement making changes in the final array.
3. Make a note of the elements that can be changed.
4. These include elements $> m$ and elements whose frequency is $> n / m$.
5. Now we just need to change these elements into the elements we want.



Implementation

```
int n, m, changeSum = 0; cin >> n >> m;

vector<int>a(n), f(m + 1);
for(auto &i : a) cin >> i;

vector<int>changes;

for(int i = 0; i < n; i++) {
    if(a[i] > m) changes.push_back(i);
    else {
        if(f[a[i]] == n / m) changes.push_back(i);
        else f[a[i]]++;
    }
}

for(int i = 1; i <= m; i++) {
    while(f[i] < n / m) {
        a[changes.back()] = i;
        changes.pop_back();
        changeSum++; f[i]++;
    }
}

cout << n / m << " " << changeSum << "\n";
for(auto &i : a) cout << i << " ";
```

Problem 3: K-Complete Word

Good!



Problem Link: <https://codeforces.com/problemset/problem/1332/C>

Word s of length n is called k -complete if

- s is a palindrome, i.e. $s_i = s_{n+1-i}$ for all $1 \leq i \leq n$;
- s has a period of k , i.e. $s_i = s_{k+i}$ for all $1 \leq i \leq n - k$.

For example, "abaaba" is a 3-complete word, while "abccba" is not.

Bob is given a word s of length n consisting of only lowercase Latin letters and an integer k , such that n is divisible by k . He wants to convert s to any k -complete word.

To do this Bob can choose some i ($1 \leq i \leq n$) and replace the letter at position i with some other lowercase Latin letter.

So now Bob wants to know the minimum number of letters he has to replace to convert s to any k -complete word.

Note that Bob can do zero changes if the word s is already k -complete.

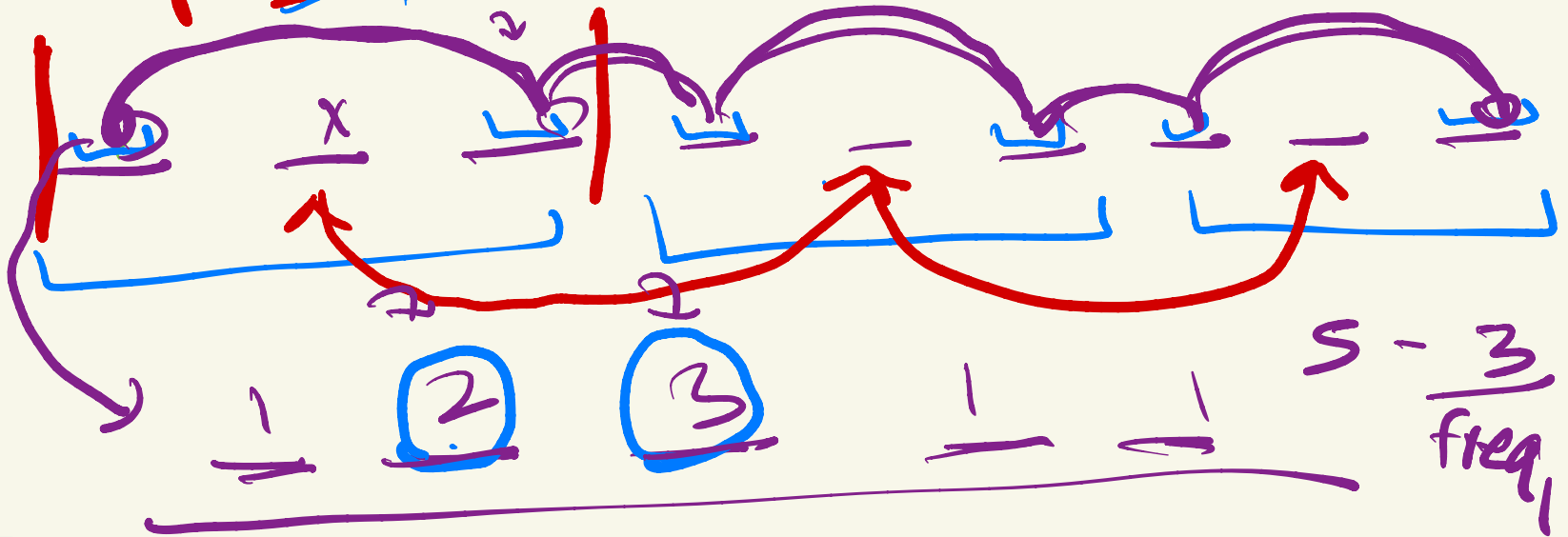
You are required to answer t test cases **independently**.

String $\Rightarrow$ K-complete

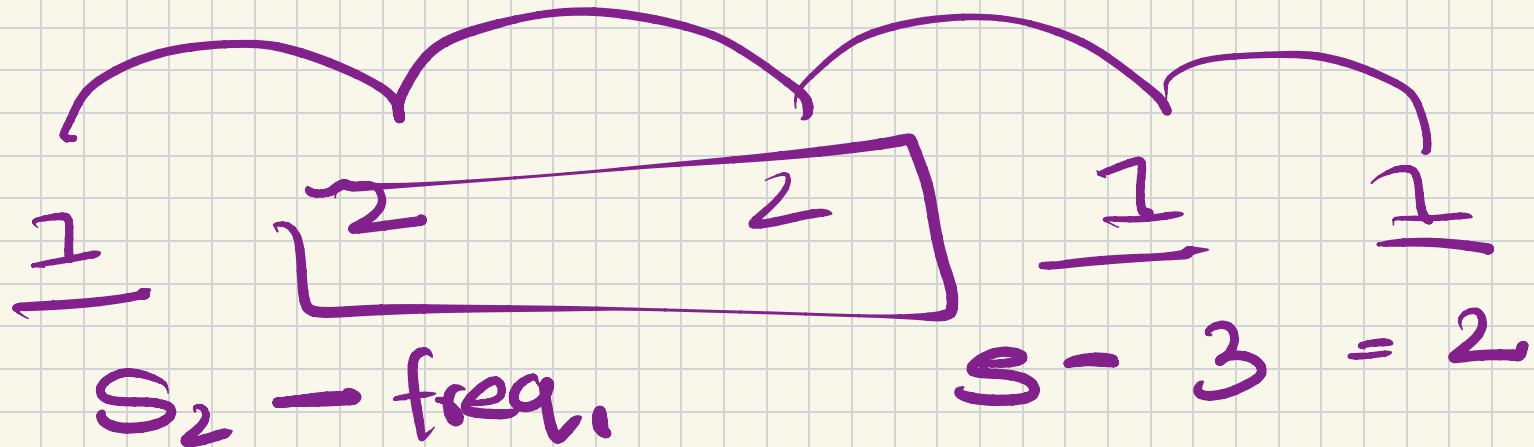
K = 3

Pen &
Paper

palindrome



For each chain
↳ $+(size\ of\ chain) - (max\ freq)$





Problem 3: K-Complete Word

Example

input	Copy
4 6 2 abaaba 6 3 abaaba 36 9 hippopotomonstrosesquippedaliophobia 21 7 wudixiaoxingxingheclp	
output	Copy
2 0 23 16	

Note

In the first test case, one optimal solution is aaaaa.

In the second test case, the given word itself is *k*-complete.



Some observations:

1. $S[0] = s[n - 1]$
2. But $s[0] = s[k - 1]$
3. This means that every period should also be a palindrome.
4. Also all periods should be the same, i.e. $s[0..k-1] = s[k..2k - 1]$

Solution



1. For every pair of elements that must be same in a period, note down the frequency of elements over all periods.
2. Now since you want to make minimum changes, you make all elements in this pairing = element with max frequency.
3. Cost = number of elements in this pairing - max frequency

Implementation



```
int tt; cin >> tt;
```

```
while(tt--) {  
    int n, k, answer = 0; cin >> n >> k;  
    string s; cin >> s;
```

```
    for(int i = 0; i < (k + 1) / 2; i++) {  
        vector<int>f(26);
```

```
        for(int j = 0; j < n; j += k) {  
            f[s[j + i] - 'a']++;  
            if(j + i != j + k - 1 - i) f[s[j + k - 1 - i] - 'a']++;  
        }
```

```
        answer += (accumulate(f.begin(), f.end(), 0) - *max_element(f.begin(), f.end()));
```

```
    }  
    cout << answer << "\n";
```

```
}
```

task

max freq

sum